



BLUE NOVA SAS

BASTION

Boveda Personal de Credenciales

MANUAL TECNICO

David Alexander Benz Zambrano

Logica de Programacion - UIDE - Paralelo 1-CIB-1A

Quito - Ecuador - 2026

Introduccion y proposito

BASTION es una boveda personal de credenciales desarrollada como proyecto integrador de la asignatura Logica de Programacion (Msc. Lilian Aman) dentro de la carrera de Ingenieria en Ciberseguridad de la Universidad Internacional del Ecuador (UIDE), paralelo 1-CIB-1A. Autor: David Alexander Benz Zambrano, bajo Blue Nova SAS.

El proposito del sistema es permitir que una persona genere contraseñas robustas y las almacene de forma local y organizada, todo protegido por una única contraseña maestra. El proyecto nace del diseño realizado en el Aprendizaje Autonomo 1 (análisis, casos de uso y diagrama de flujo) y se materializa como producto funcional aplicando las estructuras de programación vistas en clase: bucles `while` y `for`, estructuras de selección `if / elif / else`, operadores, contadores, tuplas, listas, diccionarios y funciones con parámetros por defecto.

El sistema existe en dos formas de uso complementarias:

- Version de consola: se ejecuta con `python3 bastion.py` y persiste los datos en un archivo `boveda.json` local.
- Aplicación web: publicada en GitHub Pages, ofrece la misma idea de boveda en una interfaz gráfica que vive enteramente en el navegador del usuario.

Este manual está dirigido a desarrolladores que vayan a mantener o extender el código y al docente que evalúa el entregable. Documenta el estado real del código fuente y, donde corresponde, distingue lo implementado de lo proyectado como evolución.

Nota de alcance importante para el lector

Durante la elaboración de este manual se contrastó la descripción de diseño con el código fuente realmente entregado. Existen diferencias que se documentan con honestidad técnica, porque un manual debe describir el sistema que existe:

- La versión de consola (`bastion.py`) implementa creación de boveda, inicio de sesión con control de intentos, generador, agregar, buscar, listar y cambio de maestra. **No** incluye, en el código actual, el registro de historial de accesos ni una función `registrar_acceso` ; tampoco importa `datetime` .
- La constante `CATEGORIAS` está implementada como **diccionario** (clave de menú -> nombre), no como tupla.
- La boveda en disco contiene dos claves: `hash_maestra` y `credenciales` . La clave `historial` está prevista en el diseño pero no se persiste en la versión actual.
- La aplicación web entregada es un único archivo `BASTION.html` autocontenido que funciona con JavaScript puro (sin Pyodide), usa un cifrado demostrativo XOR+Base64 y guarda la boveda en `localStorage` bajo la clave `bastion_vault` .

A lo largo del manual, los elementos descritos en el diseño pero aún no implementados se marcan claramente como Proyección / Roadmap.

Stack tecnologico

Capa / area	Tecnologia	Rol en BASTION
Lenguaje (consola)	Python 3 (solo biblioteca estandar)	Logica de negocio, generador, validaciones, persistencia
Modulos estandar	<code>hashlib</code>	Hash SHA-256 de la contraseña maestra
	<code>json</code>	Lectura y escritura de la boveda en disco
	<code>os</code>	Verificar existencia del archivo <code>boveda.json</code>
	<code>random</code> , <code>string</code>	Construir el alfabeto y elegir caracteres al azar
	<code>getpass</code>	Leer la contraseña maestra sin mostrarla en pantalla
	<code>sys</code>	Cierre limpio del programa al bloquear la boveda
Estructura (web)	HTML5	Pantalla de bloqueo, panel de boveda, modales
Estilo (web)	CSS3	Identidad Blue Nova: azul <code>#008AFC</code> , cyan <code>#97F8F4</code> , fondo navy <code>#01071A</code>
Logica (web)	JavaScript (vanilla, ES6)	Generador, cifrado demostrativo, render, persistencia
Almacenamiento web	<code>localStorage</code> del navegador	Boveda cifrada bajo la clave <code>bastion_vault</code>
Control de versiones	Git y GitHub	Repositorio github.com/Bluenovasas/bastion-uide
Publicacion	GitHub Pages	Hospedaje estatico de la version web
Proyeccion (web)	Pyodide (Python -> WebAssembly)	Reutilizar <code>bastion.py</code> dentro del navegador (roadmap)

Arquitectura en tres capas

BASTION sigue, en ambas versiones, una separacion en tres capas: presentacion, logica de negocio y persistencia. Esto facilita el mantenimiento y permite que cada capa evolucione sin romper a las demas.

Version de consola

Capa	Responsabilidad	Donde vive en el codigo
Presentacion	Menus de texto, lectura de teclado, mensajes al usuario	<code>main()</code> , <code>menu_principal()</code> , utilidades de entrada
Logica de negocio	Reglas del producto: generar, validar fortaleza, buscar, agregar, cambiar maestra	<code>generar_contrasena()</code> , <code>calcular_fortaleza()</code> , <code>agregar_credencial()</code> , <code>buscar_credencial()</code> , <code>iniciar_sesion()</code>
Persistencia	Leer y escribir la boveda en disco; hash de la maestra	<code>cargar_boveda()</code> , <code>guardar_boveda()</code> , <code>calcular_hash()</code> , <code>existe_boveda()</code>

El flujo tipico es: la capa de presentacion recoge una opcion del menu, invoca una funcion de logica de negocio, y esa funcion delega en la capa de persistencia cuando necesita guardar o cargar el archivo `boveda.json`.

Version web

Capa	Responsabilidad	Donde vive en <code>BASTION.html</code>
Presentacion	Pantalla de bloqueo, tarjetas de credenciales, modales, indicador de fortaleza	Bloque HTML/CSS y funciones de render (<code>renderCards</code> , <code>renderCats</code> , <code>renderStats</code> , <code>paintMeter</code>)
Logica de negocio	Generador con conjuntos sin caracteres ambiguos, calculo de fortaleza, filtros, desbloqueo	<code>genPassword()</code> , <code>strength()</code> , <code>doUnlock()</code> , <code>filtered()</code>
Persistencia	Cifrar/descifrar la boveda y leerla o escribirla en <code>localStorage</code>	<code>encrypt()</code> , <code>decrypt()</code> , <code>persist()</code> , <code>rawStored()</code> , <code>vaultExists()</code>

Ambas versiones comparten la misma filosofia (sin nube, una sola contrasena maestra, generador con indicador de fortaleza), pero son implementaciones independientes: la consola en Python y la web en JavaScript.

Descripcion del codigo `bastion.py`

El archivo `bastion.py` esta organizado por comentarios de seccion que marcan las tres capas. A continuacion se resume el rol de las funciones principales y de las estructuras de datos.

Estructuras de datos

- **Diccionario de la boveda.** Es la estructura central. En la version actual tiene dos claves:
 - `hash_maestra` (str): el hash SHA-256 hexadecimal de la contrasena maestra. Nunca se guarda la clave en claro.

- `credenciales` (dict): un diccionario anidado donde la clave es el nombre del servicio y el valor es otro diccionario con `categoria`, `usuario` y `contrasena`.
- `historial` (Proyeccion): el diseno contempla una tercera clave que seria una lista de accesos con fecha y hora. No esta implementada en el codigo actual.
- **CATEGORIAS.** Diccionario constante (clave del menu -> nombre de categoria) con seis valores: Redes sociales, Banca y finanzas, Trabajo, Correo, Claves API / SSH, Otros. Funciona como catalogo inmutable de referencia. En el diseno se describe como tupla; en el codigo real es un diccionario, lo que permite usar la clave numerica directamente en el menu y validar con el operador de pertenencia `in`.
- **Lista historial (Proyeccion).** El diseno la describe como una lista de registros de acceso. La estructura de lista si se usa internamente en la web para las credenciales (`state.creds`), pero el historial como tal no se persiste en la version de consola actual.

Funciones con parametros por defecto

El codigo emplea parametros por defecto para hacer funciones reutilizables:

- `pedir_entero(mensaje, minimo, maximo)`: solicita y valida un entero dentro de un rango usando un bucle `while`. Aunque recibe minimo y maximo, se usa como utilidad generica de entrada validada.
- `agregar_credencial(boveda, contrasena_sugerida=None)`: el parametro por defecto `contrasena_sugerida=None` permite reutilizar la funcion en dos escenarios: agregar una credencial desde cero, o guardar una contrasena recién generada. Si llega una sugerida, ofrece reutilizarla; si no, pide una manualmente.

Nota: el diseno menciona `registrar_acceso(boveda, evento="Acceso concedido")` como ejemplo de parametro por defecto. Esa funcion pertenece al roadmap del historial y no esta presente en el codigo actual.

Funciones principales por capa

- Persistencia: `calcular_hash(texto)` devuelve el SHA-256 hexadecimal; `existe_boveda()` comprueba con `os.path.exists`; `cargar_boveda()` y `guardar_boveda(boveda)` leen/escriben el JSON con `ensure_ascii=False` para respetar acentos.
- Utilidades de interfaz: `leer_clave_oculta(mensaje)` usa `getpass` con respaldo a `input`; `preguntar_si_no(mensaje)` resuelve respuestas s/n; `enmascarar(contrasena)` deja visibles solo el primer y ultimo caracter.
- Logica de negocio: `calcular_fortaleza(longitud, tipos_seleccionados)` clasifica en Debil/Media/Fuerte; `crear_boveda()` (CU-01); `iniciar_sesion()` (CU-02) con contador de intentos; `generar_contrasena()` (CU-03); `agregar_credencial()` (CU-04); `buscar_credencial()` (CU-06) sin distinguir mayusculas; `ver_credenciales()` (CU-07) con clave enmascarada; `cambiar_contrasena_maestra()`.

- Presentacion: `menu_principal(boveda)` y `main()` enrutan opciones con `while True` + `break` e `if / elif / else`.

El siguiente fragmento muestra el control de intentos en el inicio de sesion, comparando hashes y nunca contraseñas en claro:

```
intentos = 0
while intentos < MAX_INTENTOS:          # MAX_INTENTOS = 3
    clave = leer_clave_oculta("Ingrese su contraseña maestra: ")
    if calcular_hash(clave) == boveda["hash_maestra"]:
        return boveda                    # acceso concedido
    intentos += 1                         # contador de fallos
    print("Intentos restantes:", MAX_INTENTOS - intentos)
print("Boveda bloqueada por seguridad.")
sys.exit()
```

Como funciona la aplicacion web por dentro

La aplicacion web entregada es un unico archivo `BASTION.html` autocontenido (HTML, CSS y JavaScript en el mismo documento). Funciona sin servidor y sin dependencias externas: basta abrir la pagina publicada en GitHub Pages. A continuacion se describe su funcionamiento real.

Estado y persistencia en localStorage

El estado en memoria se mantiene en un objeto `state = { master, creds, cat, q }`. La boveda se persiste cifrada en `localStorage` bajo la clave `bastion_vault` (constante `STORE`). Cada registro guardado incluye un campo `magic` (`BASTION-V1`) que actua como marca de integridad: si al descifrar el `magic` no coincide, la contraseña es incorrecta o el dato esta corrupto.

```
const STORE = "bastion_vault";
function persist() {
    const payload = { magic: "BASTION-V1", savedAt: new Date().toISOString(), creds: state.creds };
    localStorage.setItem(STORE, encrypt(payload, state.master));
}
```

Cifrado demostrativo

El cifrado web es academico: deriva una clave a partir de la maestra mas una sal fija con una mezcla acumulativa (`keyBytes`), aplica XOR byte a byte sobre el JSON y codifica el resultado en Base64. Las funciones clave son `encrypt(obj, master)` y `decrypt(b64, master)`. El propio codigo documenta en comentario que no es criptografia de grado produccion y que el objetivo es ilustrar el concepto de una boveda cifrada.

Generador y fortaleza

`genPassword(len, opts)` construye la contraseña con conjuntos que excluyen caracteres ambiguos (sin l, O, I, 0, 1), garantiza al menos un caracter de cada conjunto activo y mezcla con `shuffle`. La aleatoriedad usa `crypto.getRandomValues` cuando esta disponible. `strength(pw)` puntua longitud y variedad para devolver Debil, Media, Fuerte o Muy fuerte, y `paintMeter` pinta la barra indicadora.

Render e interaccion

El render se hace por funciones especializadas: `renderCards` (tarjetas de credenciales), `renderCats` (categorias), `renderStats` (totales) y `renderAll` que las coordina. El desbloqueo lo maneja `doUnlock`, que segun exista o no boveda actua en modo crear o iniciar sesion (`refreshLockMode`). Hay control de copiado al portapapeles (`copyText`), modales (`openModal`, `openAddModal`), y un alternar de visibilidad de contraseña (`eyeToggle`), equivalente al check Mostrar contraseña.

Proyeccion: arquitectura Pyodide

El diseno contempla una variante en la que `bastion.py` se ejecuta dentro del navegador mediante Pyodide (Python compilado a WebAssembly): se cargaria Pyodide, se descargaria `bastion.py` al sistema de archivos virtual, se definirian funciones puente `web_*` invocadas desde JavaScript, y se mostraria cada llamada en un panel Consola Python en vivo. En el entregable actual la web no usa Pyodide; reimplementa la logica en JavaScript puro para que la pagina funcione offline tras la primera carga.

Modelo de seguridad y limitaciones

Lo que protege hoy

- **Hash de la contraseña maestra (consola).** En `bastion.py` la maestra nunca se guarda en claro: se almacena su hash SHA-256 (`calcular_hash`) y el inicio de sesion compara hash contra hash.
- **Control de intentos (consola).** El inicio de sesion permite un maximo de 3 intentos (`MAX_INTENTOS`); al agotarse, la boveda se bloquea y el programa termina con `sys.exit()`.
- **Enmascarado en pantalla.** Al listar credenciales, la contraseña se muestra ofuscada (`enmascarar`), dejando visibles solo el primer y ultimo caracter.
- **Sin nube.** Ni la consola ni la web envian datos a servidores. Los datos viven en `boveda.json` o en el `localStorage` del navegador, reduciendo la superficie de ataque. La web solo necesitaria internet la primera vez (y en el roadmap Pyodide, para descargar el runtime).

Limitaciones reconocidas

- En la consola, las credenciales se guardan en JSON sin cifrado simetrico: el hash protege la maestra, pero los valores de cada credencial quedan legibles en el archivo. Es una limitacion academica explicita.

- En la web, el cifrado XOR+Base64 es demostrativo y no debe considerarse seguro frente a un atacante real.

Proyeccion de seguridad (Roadmap)

La evolucion planificada del producto contempla:

- Cifrado autenticado **AES-256-GCM** sobre el contenido de la boveda.
- Derivacion de clave con **Argon2id** a partir de la contraseña maestra (resistencia a fuerza bruta y a hardware especializado).
- Persistencia de un **historial de accesos** con fecha y hora para auditoria.

Con ello, ni siquiera quien obtenga el archivo o el contenido de `localStorage` podria leer las credenciales sin la maestra.

Estructura de archivos del proyecto

Arbol del repositorio segun el estado entregado y la organizacion prevista en el README:

```

bastion-uide/
├─ bastion.py          Codigo fuente de la version de consola
├─ BASTION.html        Aplicacion web autocontenida (HTML+CSS+JS)
├─ boveda.json         Boveda local generada en tiempo de ejecucion (ignorada por Git)
├─ README.md           Documentacion academica del proyecto
├─ LICENSE             Licencia MIT
├─ .gitignore          Exclusiones de Git (incluye boveda.json y __pycache__)
├─ docs/              Diagramas de diseno (Autonomo 1)
│   └─ bastion_diagrama.rap Diagrama de flujo en Raptor
├─ videos/            Videos explicativos del entregable
│   └─ video1_diagrama_flujo.mp4
│   └─ video2_codigo_funcional.mp4

```

Notas:

- `boveda.json` se crea al usar el programa y esta listado en `.gitignore`, de modo que las credenciales no se suben al repositorio.
- El diseno menciona una carpeta `webapp/` con `index.html`, `styles.css`, `app.js` y `favicon.svg` para la variante Pyodide. En el entregable actual la web se concentra en el unico archivo `BASTION.html`.

Procedimiento de despliegue en GitHub Pages

La aplicacion web se publica como sitio estatico. Pasos:

1. Subir el codigo al repositorio github.com/Bluenovasas/bastion-uide (rama principal).

2. Asegurar que el archivo a servir este accesible. La forma mas simple para el archivo unico es exponer `BASTION.html` como pagina; si se desea una URL tipo `index.html`, basta copiarlo o renombrarlo a `index.html` en la raiz o en una carpeta `webapp/`.
3. En GitHub: Settings -> Pages -> Build and deployment -> Source: Deploy from a branch.
4. Seleccionar la rama (por ejemplo `main`) y la carpeta raiz (`/root`) o `/docs` segun donde este el HTML, y guardar.
5. Esperar a que GitHub Pages publique. La URL resultante para la variante con carpeta es del estilo `https://bluenovajas.github.io/bastion-uide/webapp/`.

Comandos de referencia para publicar cambios:

```
git add BASTION.html README.md
git commit -m "Publicar version web de BASTION"
git push origin main
```

Como es un sitio estatico sin construccion (build), cualquier `push` a la rama publicada actualiza el sitio en pocos minutos.

Como ejecutar el proyecto localmente

Version de consola

Requiere unicamente Python 3 (la biblioteca estandar es suficiente, no hay dependencias que instalar):

```
python3 bastion.py
```

La primera ejecucion ofrece crear la boveda; a partir de entonces aparece la opcion de iniciar sesion. La boveda queda en `boveda.json` en el mismo directorio.

Version web (servida localmente)

El archivo `BASTION.html` actual funciona incluso abriendolo directo en el navegador, porque su logica es JavaScript puro y no descarga `bastion.py`. Aun asi, se recomienda servirlo con un servidor estatico para reproducir el comportamiento de GitHub Pages y evitar restricciones del esquema `file://` (por ejemplo, en la variante Pyodide donde el navegador debe descargar `bastion.py` al sistema de archivos virtual, lo cual exige servirlo por HTTP):

```
python3 -m http.server 8000
```

Luego abrir `http://localhost:8000/BASTION.html` (o la ruta correspondiente si se ubico en una carpeta `webapp/`). Servir por HTTP es obligatorio para la futura variante con Pyodide y buena practica general para la web.

Resumen

Aspecto	Consola (<code>bastion.py</code>)	Web (<code>BASTION.html</code>)
Lenguaje	Python 3 estandar	HTML + CSS + JavaScript
Maestra	Hash SHA-256, sin texto claro	Clave derivada + verificacion por <code>magic</code>
Credenciales	JSON sin cifrar (limitacion academica)	Cifrado demostrativo XOR+Base64
Persistencia	<code>boveda.json</code> local	<code>localStorage</code> clave <code>bastion_vault</code>
Intentos	Maximo 3, luego bloqueo	Verificacion al desbloquear
Pyodide	No aplica	No (roadmap)
Historial	No implementado (roadmap)	No implementado (roadmap)
Cifrado fuerte	AES-256-GCM + Argon2id (roadmap)	AES-256-GCM + Argon2id (roadmap)

Este manual describe el sistema tal como esta implementado y senala con claridad las funciones proyectadas, de modo que cualquier desarrollador pueda mantener el codigo actual y el docente pueda evaluar tanto lo construido como la vision de evolucion del producto.